

Backend & API Implementation Report: Shift & Workforce Scheduling Module

1. Overview

This report outlines the technical requirements for the backend and API teams to transition the Shift & Workforce Scheduling module from a frontend prototype with seeded data to a production-ready system. The backend must handle complex business logic, including labor compliance, fairness algorithms, and real-time synchronization.

2. Database Architecture (PostgreSQL & Sequelize)

The system requires a relational database structured to support four integrated submodules. Key tables include:

- **User Profiles:** `Employees/Users` (roles, license classes), `Skill_Taxonomy` (hierarchical skills), and `Employee_Skills` (proficiency mapping).
- **Rostering:** `Rosters/Schedules` (5-in/5-out logic), `University_Events` (bulk creation), and `Zoning_Assignments`.
- **Leave & Availability:** `Leave_Requests` (status tracking), `Leave_Balances` (accrual logic), and `RealTime_Availability`.
- **Compliance & Finance:** `Overtime_Records` (hours and rates), `Fairness_Metrics` (seniority and history data), `Compliance_Logs` (rest period/hour violations), and `Audit_Trail` (system-wide change logs).

3. API Endpoint Requirements

The API team must implement **RESTful endpoints** to perform CRUD (Create, Read, Update, Delete) operations across all scheduling resources.

- **Roster Endpoints:** Handle drag-and-drop swaps and bulk event-driven shift creation.
- **Leave Workflow:** Endpoints for submitting requests and supervisor approval/rejection cards.
- **Journey Tracking:** Routes to store journey distance (km), destinations, and automatically generated audit references (e.g., PAY-YYYYMMDD-ID).
- **Mobile Interface:** Optimized endpoints for the "one-tap" driver status updates to minimize data usage.

4. Core Business Logic & Engines

The backend team is responsible for implementing the "Intelligence Layer" through the following services:

- **Compliance Engine:** Automatically validate every roster change against the **Zimbabwe Labor Act**. It must trigger **Errors** for exceeding 45 hours/week and **Warnings** for rest periods under 11 hours.
- **Fairness Algorithm:** Implement a two-level sort for overtime distribution, prioritizing drivers with the **fewest recent overtime hours** followed by **highest seniority**.
- **AI Skill Matching:** A Python-based service using **cosine similarity** to rank drivers against task requirement vectors (e.g., garden work, brush cutting).
- **Allowance Calculator:** Automate the ES-A supplement (+\$8) for shift starts before **05:30 AM** and categorize packages (LOC, LD, CB) based on captured distance.

5. Real-Time & Security Integration

- **Socket.io:** Implement a WebSocket server to handle **real-time GPS pings** for the live map and instant status updates on the availability board.
- **Authentication:** Integrate with the P1 module to enforce **Role-Based Access Control (RBAC)**. Routes must be secured via JWT, differentiating between Supervisor (edit), Driver (read-only), and Admin (system-wide) roles.
- **Notifications:** Wire the stub-ready notification layer to **Firebase Cloud Messaging** and **BulkSMS Zimbabwe** for 24-hour shift alerts and fatigue warnings.

6. Development & Git Workflow

To ensure system stability, the team must follow these protocols:

- **Branching:** Never push directly to the **main** branch; use feature-specific branches (e.g., `git checkout -b feature-compliance-logic`).
- **Pull Requests:** All API changes must undergo team review via **Pull Requests** to avoid overwriting shared contracts or breaking integration points.
- **Database Migrations:** Use Sequelize to ensure consistent schema updates across all developer environments.